SQL PROJECT

Vehicle Insurance Management System

Project Requirements and Database Design

Project	Vehicle Insurance Management System (VIMS)
Database	`vims` (MySQL)
Tables	7 tables, 501 rows in total
Questions	20 analysis requirements (7 Beginner, 7 Intermediate, 6 Advanced)
Reference date	2026-10-20
Files	VIMS_Dataset.xlsx (data), vims_setup.sql (create tables + load data), this document, VIMS Master Answers.docx

1. Project Overview

A vehicle insurance company sells insurance policies for two-wheelers, cars, SUVs and commercial vehicles through its agents. Customers register their vehicles, buy a policy under one of the insurance plans, pay the premium and may raise claims when an incident happens.

In this project you act as the data analyst of the company. You will create the database, load the data and answer **20 business questions** using SQL. The questions start with simple filtering and grow step by step to joins, subqueries, CTEs and window functions.

2. Dataset and Setup

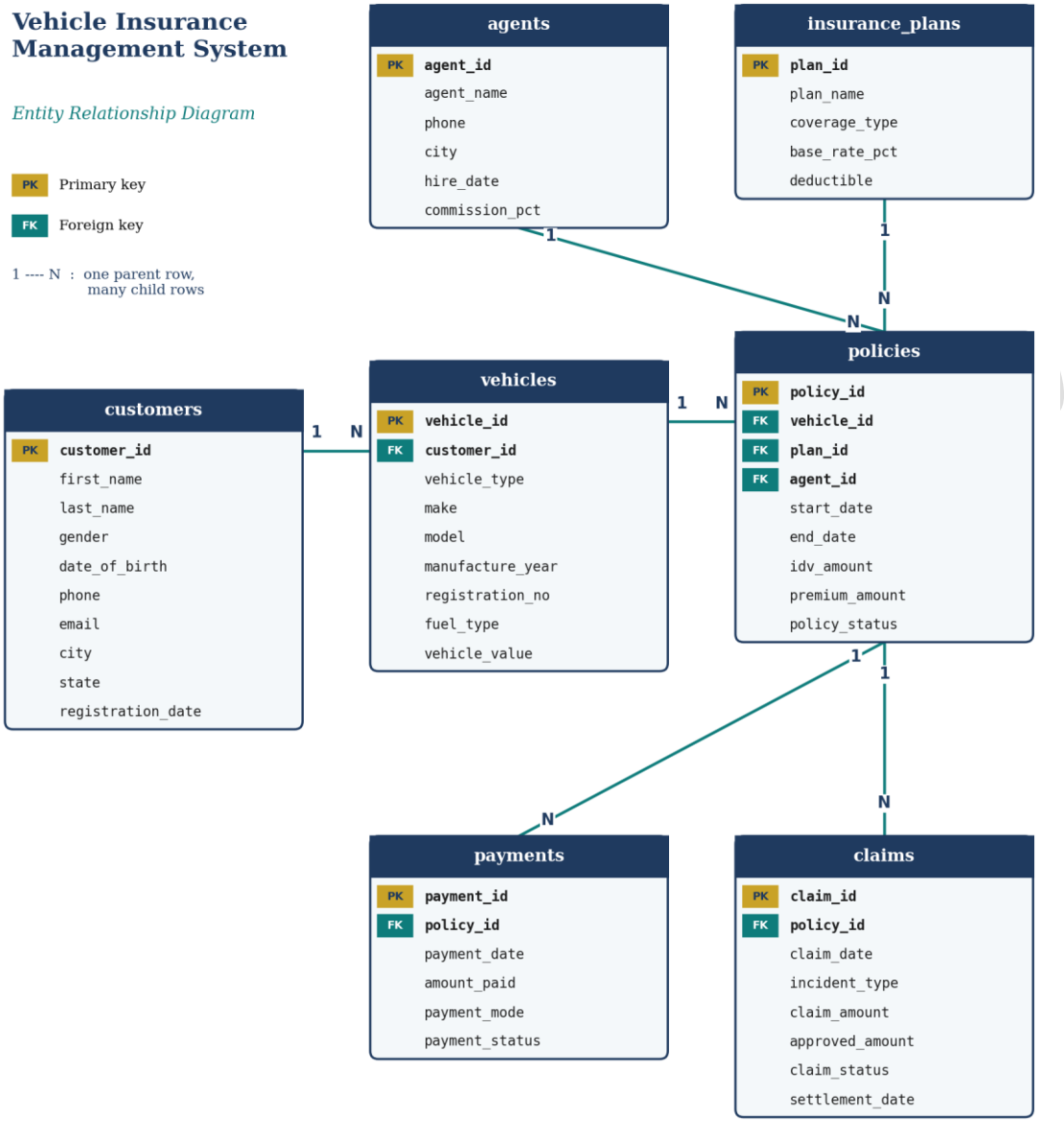
The same data is supplied in two forms. Use whichever suits your class:

- **VIMS_Dataset.xlsx** - one sheet per table. Use it to explore the data or to import with the MySQL Workbench Table Data Import Wizard.
- **vims_setup.sql** - creates the `vims` database, all tables with primary and foreign keys, and inserts all rows. Run it once and you are ready to start.

> **Loading order**
> Parent tables must be loaded before child tables: `insurance_plans`, `agents`, `customers`, `vehicles`, `policies`, then `payments` and `claims`. The setup script already follows this order.

3. Database Design

3.1 Entity Relationship Diagram



3.2 Table Structures

Primary keys (PK) uniquely identify each row. Foreign keys (FK) connect a child table to its parent table.

`insurance_plans` 6 rows

Master list of insurance plans with the base premium rate and the deductible.

Column	Data type	Key / Constraint	Description
plan_id	VARCHAR(5)	PK	Unique plan code (P01 - P06)
plan_name	VARCHAR(50)	NOT NULL	Name of the plan
coverage_type	VARCHAR(20)	NOT NULL	Third Party / Own Damage / Comprehensive
base_rate_pct	DECIMAL(4,2)	NOT NULL	Premium rate as a percentage of the IDV
deductible	INT	DEFAULT 0	Amount (Rs.) the customer bears in every claim

`agents` 15 rows

Insurance agents who sell the policies.

Column	Data type	Key / Constraint	Description
agent_id	VARCHAR(5)	PK	Unique agent code (A001 - A015)
agent_name	VARCHAR(50)	NOT NULL	Full name of the agent
phone	VARCHAR(15)		Contact number
city	VARCHAR(30)		City where the agent works
hire_date	DATE		Date the agent joined
commission_pct	DECIMAL(4,1)	NOT NULL	Commission percentage on premium sold

`customers` 80 rows

Policy holders with their personal and contact details.

Column	Data type	Key / Constraint	Description
customer_id	VARCHAR(5)	PK	Unique customer code (C001 - C080)
first_name	VARCHAR(30)	NOT NULL	First name
last_name	VARCHAR(30)	NOT NULL	Last name
gender	VARCHAR(10)		Male / Female
date_of_birth	DATE		Date of birth
phone	VARCHAR(15)		Mobile number
email	VARCHAR(60)	NULL allowed	Email address (missing for a few customers)
city	VARCHAR(30)		City of residence
state	VARCHAR(30)		State of residence
registration_date	DATE		Date the customer registered

vehicles 100 rows

Vehicles owned by customers. One customer can own many vehicles.

Column	Data type	Key / Constraint	Description
vehicle_id	VARCHAR(5)	PK	Unique vehicle code (V001 - V100)
customer_id	VARCHAR(5)	FK	Owner. References customers(customer_id)
vehicle_type	VARCHAR(20)	NOT NULL	Two-Wheeler / Car / SUV / Commercial
make	VARCHAR(30)	NOT NULL	Manufacturer (Maruti Suzuki, Tata, Honda ...)
model	VARCHAR(30)	NOT NULL	Model name
manufacture_year	INT	NOT NULL	Year of manufacture
registration_no	VARCHAR(15)	UNIQUE	RTO registration number
fuel_type	VARCHAR(15)		Petrol / Diesel / Electric
vehicle_value	DECIMAL(12,2)	NOT NULL	Current market value (Rs.)

policies 110 rows

Insurance policies issued for vehicles. A renewal creates a new policy row.

Column	Data type	Key / Constraint	Description
policy_id	VARCHAR(8)	PK	Unique policy number (POL0001 ...)
vehicle_id	VARCHAR(5)	FK	Insured vehicle. References vehicles(vehicle_id)
plan_id	VARCHAR(5)	FK	Plan taken. References insurance_plans(plan_id)
agent_id	VARCHAR(5)	FK	Selling agent. References agents(agent_id)
start_date	DATE	NOT NULL	Policy start date
end_date	DATE	NOT NULL	Policy end date (one year term)
idv_amount	DECIMAL(12,2)	NOT NULL	Insured Declared Value (Rs.)
premium_amount	DECIMAL(12,2)	NOT NULL	Annual premium = IDV x base_rate_pct / 100
policy_status	VARCHAR(15)	NOT NULL	Active / Expired / Cancelled

payments 120 rows

Premium payments against policies (single payment or two instalments).

Column	Data type	Key / Constraint	Description
payment_id	VARCHAR(8)	PK	Unique payment number (PAY0001 ...)
policy_id	VARCHAR(8)	FK	References policies(policy_id)
payment_date	DATE	NOT NULL	Date of the payment attempt
amount_paid	DECIMAL(12,2)	NOT NULL	Amount of this payment (Rs.)
payment_mode	VARCHAR(15)	NOT NULL	UPI / Credit Card / Debit Card / Net Banking / Cash / Cheque
payment_status	VARCHAR(10)	NOT NULL	Success / Failed / Pending

`claims` 70 rows

Insurance claims raised against policies.

Column	Data type	Key / Constraint	Description
claim_id	VARCHAR(8)	PK	Unique claim number (CLM0001 ...)
policy_id	VARCHAR(8)	FK	References policies(policy_id)
claim_date	DATE	NOT NULL	Date the claim was raised
incident_type	VARCHAR(25)	NOT NULL	Accident / Theft / Fire / Natural Calamity / ...
claim_amount	DECIMAL(12,2)	NOT NULL	Amount claimed (Rs.)
approved_amount	DECIMAL(12,2)	DEFAULT 0	Amount approved by the insurer (Rs.)
claim_status	VARCHAR(15)	NOT NULL	Settled / Approved / Under Review / Rejected
settlement_date	DATE	NULL allowed	Date the claim was paid (NULL until settled)

3.3 Relationships

Parent table	Child table	Type	Join column	Meaning
customers	vehicles	1 : N	customer_id	A customer can own many vehicles
vehicles	policies	1 : N	vehicle_id	A vehicle can have many policies (renewals)
insurance_plans	policies	1 : N	plan_id	A plan is used by many policies
agents	policies	1 : N	agent_id	An agent sells many policies
policies	payments	1 : N	policy_id	A policy can be paid in instalments
policies	claims	1 : N	policy_id	A policy can have many claims

3.4 Business Rules and Data Notes

- All amounts are in Indian Rupees (Rs.) and all dates are in YYYY-MM-DD format.
- The reference date of the project is **2026-10-20**. Use this fixed date in your queries instead of CURDATE() so that your output matches the answer file.
- IDV (Insured Declared Value) is between 92% and 100% of the vehicle value. Premium = IDV x base_rate_pct / 100.
- Every policy runs for one year. A renewal is stored as a new policy row for the same vehicle. Policy status is Active, Expired or Cancelled.
- A policy is paid either in one payment or in two instalments. Cancelled policies and the four newest policies have no payment rows.
- Claim status is Settled (paid, has a settlement_date), Approved (approved but not yet paid), Under Review or Rejected (approved_amount = 0).
- Some data is deliberately incomplete to practise NULL handling and outer joins: 6 customers have no email, 6 customers own no vehicle, 10 vehicles never had a policy.

VRajasha

4. Analysis Requirements

Write one SQL query for each requirement. Compare your result with the expected columns and number of rows given below each question, then check the full answer in the master answer file.

No.	Topic	SQL concepts tested
Q1	Gmail customers from Pune and Mumbai	SELECT, WHERE, IN, LIKE, CONCAT
Q2	Five most valuable vehicles	ORDER BY, LIMIT
Q3	Vehicles by type	GROUP BY, COUNT, AVG, ROUND
Q4	Mid-range premium policies	BETWEEN, ORDER BY
Q5	Customers with missing email	IS NULL
Q6	Specific makes and fuel type	IN, AND, comparison operators
Q7	Policies expiring soon	Date functions: DATEDIFF, DATE_ADD
Q8	Customer and vehicle details	INNER JOIN
Q9	Complete policy report	Multi-table JOIN (5 tables)
Q10	Collection by payment mode	GROUP BY, SUM, COUNT, WHERE on status
Q11	Claims by incident type	GROUP BY, HAVING
Q12	Customers without any vehicle	LEFT JOIN with IS NULL (anti-join)
Q13	Vehicles never insured	LEFT JOIN, anti-join, multi-table
Q14	Agent performance and commission	LEFT JOIN, GROUP BY, arithmetic, COALESCE
Q15	Claim approval ratio per plan	CASE WHEN, conditional aggregation, JOIN
Q16	Customers owning multiple vehicles	JOIN, GROUP BY, HAVING
Q17	Above-average premium within plan	Correlated subquery
Q18	Agent ranking within city	Window function: RANK() OVER (PARTITION BY ...), CTE
Q19	Monthly premium with running total	DATE_FORMAT, GROUP BY, SUM() OVER (ORDER BY ...)
Q20	Customer loss ratio	Multiple CTEs, COALESCE, CASE, NULLIF

Q1. Display the full name, city and email of all customers who live in Pune or Mumbai and use a Gmail address.

Tables used: `customers`

Concepts: SELECT, WHERE, IN, LIKE, CONCAT

Expected result: 6 rows with columns `customer_name`, `city`, `email`

Q2. List the registration number, make, model and vehicle value of the 5 most expensive vehicles.

Tables used: `vehicles`

Concepts: ORDER BY, LIMIT

Expected result: 5 rows with columns `registration_no`, `make`, `model`, `vehicle_value`

Q3. Find the number of vehicles and the average vehicle value for each vehicle type.

Tables used: `vehicles`

Concepts: GROUP BY, COUNT, AVG, ROUND

Expected result: 4 rows with columns `vehicle_type`, `total_vehicles`, `avg_value`

Q4. Show all policies whose premium is between Rs. 30,000 and Rs. 60,000 (inclusive), highest premium first.

Tables used: `policies`

Concepts: BETWEEN, ORDER BY

Expected result: 9 rows with columns `policy_id`, `vehicle_id`, `plan_id`, `premium_amount`, `policy_status`

Q5. Find the customers whose email address is not recorded. Show customer id, name and city.

Tables used: `customers`

Concepts: IS NULL

Expected result: 6 rows with columns `customer_id`, `customer_name`, `city`

Q6. List all Petrol vehicles of Maruti Suzuki, Hyundai or Tata that were manufactured in 2020 or later, with model and manufacture year.

Tables used: `vehicles`

Concepts: IN, AND, comparison operators

Expected result: 8 rows with columns `vehicle_id`, `make`, `model`, `manufacture_year`, `fuel_type`

Q7. As of 2026-10-20, list all Active policies that will expire within the next 90 days, with the number of days left.

Tables used: `policies`

Concepts: Date functions: DATEDIFF, DATE_ADD

Expected result: 8 rows with columns `policy_id`, `vehicle_id`, `end_date`, `days_left`

Note: *The reference date 2026-10-20 is used instead of CURDATE() so that every student gets the same output.*

Q8. Show each customer's name together with the registration number, make and model of the vehicles they own. Sort by customer name and show only the first 15 rows.

Tables used: `customers`, `vehicles`

Concepts: INNER JOIN

Expected result: 15 rows with columns `customer_name`, `registration_no`, `make`, `model`

Q9. Prepare a policy report showing policy id, customer name, vehicle registration number, plan name, agent name and premium for all Active policies. Show only the top 10 by premium.

Tables used: `policies`, `vehicles`, `customers`, `insurance_plans`, `agents`

Concepts: Multi-table JOIN (5 tables)

Expected result: 10 rows with columns `policy_id`, `customer_name`, `registration_no`, `plan_name`, `agent_name`, `premium_amount`

Q10. Calculate the total premium collected (successful payments only) and the number of payments for each payment mode.

Tables used: `payments`

Concepts: GROUP BY, SUM, COUNT, WHERE on status

Expected result: 6 rows with columns `payment_mode`, `no_of_payments`, `total_collected`

Q11. For each incident type show the number of claims, total claimed amount and total approved amount. Show only incident types having at least 3 claims.

Tables used: `claims`

Concepts: GROUP BY, HAVING

Expected result: 5 rows with columns `incident_type`, `total_claims`, `total_claimed`, `total_approved`

Q12. Find the customers who have not registered any vehicle yet.

Tables used: `customers`, `vehicles`

Concepts: LEFT JOIN with IS NULL (anti-join)

Expected result: 6 rows with columns `customer_id`, `customer_name`, `city`

Q13. Find the vehicles that have never had a policy. Show registration number, make, model and owner name.

Tables used: `vehicles`, `customers`, `policies`

Concepts: LEFT JOIN, anti-join, multi-table

Expected result: 10 rows with columns `vehicle_id`, `registration_no`, `make`, `model`, `owner_name`

Q14. For every agent show the number of policies sold, total premium and the commission earned (premium x commission %). Agents with no sales must also appear.

Tables used: `agents`, `policies`

Concepts: LEFT JOIN, GROUP BY, arithmetic, COALESCE

Expected result: 15 rows with columns `agent_id`, `agent_name`, `city`, `policies_sold`, `total_premium`, `commission_earned`

Q15. For each insurance plan find the total claims, approved claims (Settled or Approved), rejected claims and the approval percentage.

Tables used: `insurance_plans`, `policies`, `claims`

Concepts: CASE WHEN, conditional aggregation, JOIN

Expected result: 6 rows with columns `plan_name`, `total_claims`, `approved_claims`, `rejected_claims`, `approval_pct`

Q16. Find customers who own more than one vehicle. Show the number of vehicles and the total value of their vehicles, and display only the top 10 customers (most vehicles first).

Tables used: `customers`, `vehicles`

Concepts: JOIN, GROUP BY, HAVING

Expected result: 10 rows with columns `customer_id`, `customer_name`, `vehicles_owned`, `total_vehicle_value`

Q17. List the policies whose premium is higher than the average premium of their own plan. Show the plan average next to each policy, sorted by plan and premium (highest first), and display only the first 12 rows.

Tables used: `policies`

Concepts: Correlated subquery

Expected result: 12 rows with columns `policy_id`, `plan_id`, `premium_amount`, `plan_avg_premium`

Q18. Rank the agents inside their own city by total premium sold (highest = rank 1). Cancelled policies must be ignored.

Tables used: `agents`, `policies`

Concepts: Window function: RANK() OVER (PARTITION BY ...), CTE

Expected result: 15 rows with columns `city`, `agent_name`, `total_premium`, `city_rank`

Q19. Show month-wise premium booked (by policy start month) for the year 2025 along with the cumulative running total.

Tables used: `policies`

Concepts: DATE_FORMAT, GROUP BY, SUM() OVER (ORDER BY ...)

Expected result: 12 rows with columns `policy_month`, `policies_issued`, `monthly_premium`, `running_total`

Q20. Calculate the loss ratio for every customer = (total approved claim amount / total premium of their policies) x 100. Show the 10 customers with the highest loss ratio and flag customers above 100% as 'High Risk'.

Tables used: `customers`, `vehicles`, `policies`, `claims`

Concepts: Multiple CTEs, COALESCE, CASE, NULLIF

Expected result: 10 rows with columns `customer_id`, `customer_name`, `total_premium`, `total_approved`, `loss_ratio_pct`, `risk_flag`